part02_architecture — Google ADK 아키텍처 및 이벤트 생명주기

> 강의노트 **Part 2 · 02. Google ADK 아키텍처** (슬라이드 16–29)와 짝을 이룬다.

Google ADK(Agent Development Kit)의 핵심 실행 아키텍처인 **Runner**, **Agent**, **Event Stream**, 그리고 **이중 계층 메모리(Session 단기 vs Memory 장기)**를 이해하고 다양한 실행 방식(표준 ADK CLI/Web, Direct Runner, FastAPI 웹 서비스)으로 검증하는 실습 모음입니다.

---

1. 디렉터리 구성 및 서브 모듈 요약

```
part02_architecture/
├── adk-based_Agent/                # ADK 표준 규격 에이전트 모음 (adk web / adk run)
│   ├── chat_time_weather_agent/    # 🌦️ 전 세계 시간·날씨 조회 및 범용 대화 에이전트
│   ├── memory_layers_agent/        # 🧠 이중 계층 메모리(Session State + Memory Service) 에이전트
│   └── README-KR.md                # adk-based_Agent 통합 가이드
├── Direct_ADK/                     # ⚙️ Runner 직접 제어 및 확장 실습 스크립트 모음
│   ├── _path.py                    # 공통 패키지(common) 및 경로 초기화 헬퍼
│   ├── direct-adk-simple-*.py      # 기초 실행: 동기(Sync) vs 비동기(Async) 비교
│   ├── run_events.py               # 🔧 요청 한 번의 전체 Event 생명주기 관찰
│   ├── run_workflow_events.py      # 🟢 LLM 없는 워크플로 노드 분기 및 이벤트 관찰
│   ├── run_memory_layers.py        # 🟢/🔧 Session(단기) vs Memory(장기) 구조 비교
│   ├── direct-agent-ref-session.py  # InMemoryMemoryService 다중 세션 기억 회상
│   ├── direct-adk-working_memory*.py# ToolContext.state 활용 작업 기억 및 대화형 챗봇
│   ├── direct-adk-agent_mod*.py    # 실시간 Open-Meteo 연동 및 LLM 컨텍스트 시각화(Plugin)
│   ├── direct-adk-fastapi-*.py     # 에이전트의 REST API 서버(FastAPI) 및 CLI 클라이언트 확장
│   ├── Session_management/         # Runner.rewind_async() 기반 세션 상태 롤백 실습
│   └── README-KR.md                # Direct_ADK 15종 스크립트 상세 가이드
└── README-KR.md                    # 본 문서 (통합 요약 가이드)
```

---

2. 서브 폴더별 핵심 실습 내용

① `adk-based_Agent/` — 표준 ADK 에이전트 패키지

Google ADK의 표준 인터페이스(`adk run` , `adk web`) 및 `AgentLoader` 규격을 준수하는 모듈식 에이전트입니다.

- `chat_time_weather_agent` (상세 문서):
  - **핵심 기능**: Open-Meteo API 연동을 통한 도시 지오코딩(`geocode`), IANA 시간대 기반 현지 시각(`get_current_time`), WMO 기상 코드 기반 실시간 날씨(`get_weather`) 조회.
  - **지능형 라우팅**: 시간·날씨 질의는 도구를 자동 호출하고, 일반 상식/코딩/일상 대화는 LLM이 직접 응답. 오프라인 폴백 DB 내장.
- `memory_layers_agent` (상세 문서):
  - **State 접두사 스코프**: 세션 간 유지되는 `user:<key>` (사용자 스코프), 단일 세션 전용 `key` (세션 스코프), 전역 `app:<key>` , 일회성 `temp:<key>` .
  - **지시문 템플릿 치환**: `{user:name?}` , `{trip?}` 형태로 프롬프트에 단기 State 자동 주입.
  - **장기 기억 연동**: `preload_memory` (매 턴 자동 사전 검색 주입) 및 `load_memory` (에이전트 능동 검색 도구) 활용.

② `Direct_ADK/` — 코드 기반 직접 Runner 제어 및 확장

ADK CLI 없이 Python 코드에서 `Runner` , `SessionService` , `MemoryService` 를 직접 조립하여 세밀하게 제어합니다 (상세 문서).

1. **동기 vs 비동기 실행 (`direct-adk-simple-sync.py` , `direct-adk-simple-async.py`)**:
    - `runner.run()` (동기 이터레이터)과 `runner.run_async()` (비동기 제너레이터)의 구조 및 블로킹/논블로킹 처리 비교.
2. `direct-agent-reference.py` ,.
    - 단일 요청에 대한 `① 도구 호출 요청 → ② Runner 도구 실행 → ③ 최종 응답 합성` 3단계 이벤트 흐름 추적.
    - ADK 2.0 Workflow의 노드 라우팅(`route`) 및 메타데이터(`node_info` , `output`) 전파 관찰.
3. **세션 & 메모리 계층 제어 (`run_memory_layers.py` , `direct-agent-ref-session.py` , `Session_management/`)**:
    - 단기 세션 State와 장기 기억의 저장 범위, 검색(단어 일치 vs 벡터 검색), 생명주기 비교.
    - `Runner.rewind_async()` 를 통한 특정 시점으로의 세션 상태 롤백(Undo) 구현.
    - `ToolContext.state` 를 활용한 작업 기억(예약 상태 누적) 챗봇.
4. **실전 확장 & 관측 (`direct-adk-agent_mod-context.py` , `direct-adk-fastapi-*.py`)**:
    - `BasePlugin` 을 활용해 LLM에 전달되는 실제 Context Window(시스템 프롬프트, 도구 스키마, 대화 이력) 및 토큰 사용량 실시간 관찰.
    - 에이전트를 FastAPI 기반 비동기 REST API 서버로 래핑하고 터미널 클라이언트와 통신.

---

3. 핵심 실행 방법 요약

① 표준 인터페이스 (`adk-based_Agent`)

```
cd part02_architecture/adk-based_Agent

# 웹 UI 실행 (http://localhost:8008)
adk web --port 8008

# CLI 대화형 실행
adk run chat_time_weather_agent
adk run memory_layers_agent
```

② 핵심 관찰 스크립트 (`Direct_ADK`)

```
cd part02_architecture/Direct_ADK

# 1. Event 라이프사이클 관찰 (🔧 Gemini 키 필요)
uv run run_events.py

# 2. Workflow 노드 분기 이벤트 관찰 (🟢 키 불필요)
uv run run_workflow_events.py

# 3. 이중 계층 메모리(Session vs Memory) 비교 (🟢 오프라인 / 🔧 LLM)
uv run run_memory_layers.py offline
uv run run_memory_layers.py llm

# 4. LLM 전체 컨텍스트 윈도우 모니터링
uv run direct-adk-agent_mod-context.py

# 5. FastAPI 에이전트 서버 구동
uv run direct-adk-fastapi-server.py
```